

Spring AOP 原理和实现—复习要点

目录

1 AOP 思想概述 (Aspect Oriented Programming Overview)

1.1 为什么需要 AOP? (Why AOP?)

- OOP（面向对象编程）通过封装、继承、多态建立对象层次结构，但在以下场景存在局限性[p.6]
- 当需要在分散的、不具有继承层次的对象中引入公共行为时，OOP 无法避免代码重复 [p.6]
- OOP 允许定义从上到下的关系，但不适合定义从左到右的横切关系 [p.6]
- 典型案例：日志记录（Logging）、性能监控（Performance Monitoring）、安全认证（Security）、事务控制（Transaction）等——这些代码分散在所有对象层次中，与核心业务功能关系不大[p.6]
- 传统解决方案（纵向继承）带来的问题 [p.8–9]
 - 代码重复：每个业务方法都需编写相同的日志/权限代码 [p.9]
 - 耦合度高：横切逻辑与核心业务紧密耦合，修改影响面大 [p.9]
 - 难以维护：跨模块实现一致更新困难 [p.9]
- AOP 解决思路：通过切面（Aspect）将横切逻辑抽取出来，统一织入，保持业务代码简洁，方便横切逻辑的修改和复用 [p.9]

1.2 什么是 AOP? (What is AOP?)

- AOP（Aspect Oriented Programming，面向切面编程）：一种程序设计思想，是对 OOP 的完善和补充 [p.10]
- 起源：1990 年，PARC（Xerox Palo Alto Research Center）研究人员提出，用于减少代码重复，提高开发效率 [p.10]
- 核心机制：采取横向抽取（Horizontal Extraction），取代传统的纵向继承体系解决重复性代码问题 [p.10]

1.3 核心概念 (Core Concepts)

- 关注点（Concern）：一个特定的问题、概念或应用程序需要达到的目标 [p.11]
 - 核心关注点（Core Concerns）：完成核心业务逻辑的关注点 [p.11]

- 横切关注点 (Cross-cutting Concerns): 实现代码散落在很多类或方法中的关注点 (日志、安全等) [p.11]
- 切面 (Aspect): 对横切关注点的模块化, 将散落的横切代码归整在一起 [p.11]
 - 一般包含切点 (Pointcut) 定义和通知/建议 (Advice) 方法 [p.11]
- 动态横切 (Dynamic Crosscutting): 在程序运行时, 通过切入点匹配连接点, 执行特定通知来增强代码 [p.12]
 - 核心技术: 连接点、切入点、通知/建议、切面 [p.12]
 - 应用场景: 安全验证、日志记录等——基本代表了 AOP 的主流使用 [p.12]
- 静态横切 (Static Crosscutting): 在编译时或编译后, 不改变原始代码, 向类中引入新成员 (字段、方法) 或修改继承关系 [p.12]
 - 通过引入 (Introduction) / 混入 (Mixin) 修改对象结构, 扩展功能 [p.12]
 - 应用场景较少, 只能使用 AspectJ 实现 [p.12]
- 连接点 (Join Point): 程序执行过程中的中断点, 可以是方法调用 (Method Invocation)、字段访问 (Field Access) 或异常抛出 (Throws) [p.13]
 - Spring AOP 仅支持方法调用作为连接点 [p.13]
 - 连接点是能够插入切面的候选点集合 [p.13]
- 切点 (Pointcut): 从候选连接点中选择的确定的插入切面的连接点, Spring AOP 中用表达式选择 [p.13]
- 通知/建议 (Advice): 在切点上运行切面的方式, 分为前置、后置、环绕、异常、最终五种 [p.13]
- 织入 (Weaving): 将切面应用到原始/目标对象的方式, 分为动态织入/横切和静态织入/横切 [p.13]

2 AOP 的实现方式 (AOP Implementation Approaches)

2.1 三种实现原理

- 编译期 / 语言扩展 (Compile-time): 由编译器将切面增加到字节码, 需定义新关键字和扩展编译器 [p.16]
 - 具体实现: AspectJ 框架等 [p.16]
- 类加载器 (Class Loader): 在目标类被装载到 JVM 时, 通过特殊的类加载器对目标类的字节码重新“增强” [p.16]
- 运行时 / 动态代理 (Runtime / Dynamic Proxy): 目标对象和切面都是普通 Java 类, 通过 JVM 动态代理功能或第三方库实现运行期动态织入 [p.16]
 - 具体实现: JDK 动态代理、CGLIB 字节码代理、AspectJ 代理等 [p.16]

2.2 JDK 动态代理 (JDK Dynamic Proxy)

- 基于 Java 反射机制的运行时代理 [p.17–21]
- 要求目标 Bean 必须实现一个或多个接口 [p.30]
- 通过 `java.lang.reflect.Proxy` 和 `InvocationHandler` 接口实现 [p.17]

2.3 AspectJ 实现

- AspectJ 是基于 Java 语言的 AOP 框架，解决系统级横切关注点的模块化问题 [p.22]
- 通过专门的编译器（ajc, AspectJ Compiler）或类加载器在编译期、编译后或加载时将切面代码织入字节码 [p.22]
- 具有强大的静态和动态织入能力，是功能最完整的 AOP 解决方案 [p.22]
- 缺点：需要引入第三方库，学习新语法成本较高 [p.22]
- 静态横切只能使用 AspectJ 实现 [p.22]

2.3.1 AspectJ 实现静态横切示例

- 通过引入（Introduction）/混入（Mixin）为已有类添加新接口和新方法，不修改原类代码 [p.24–25]
- 示例：为 Product 类混入待售状态判断功能和 Comparable 排序功能 [p.24–27]
- 非 AOP 替代方案：修改类的硬编码、使用适配器模式重构 [p.24]

3 Spring AOP 的使用 (Spring AOP Usage)

3.1 Spring AOP 简介

- Spring 框架支持多种 AOP 实现：JDK 代理、CGLIB 字节码生成代理和 AspectJ 等 [p.29]
- 在业务应用中一般只涉及动态横切，不推荐基于 AspectJ 的静态横切 [p.29]
- 从 Spring 3.x 起，建议使用 AspectJ 的方式实现 AOP [p.29]
- AspectJ AOP 在 Spring 中分为 XML 配置和注解两种方式 [p.29]

3.2 代理模式 (Proxy Patterns)

- **JDK 动态代理**：当目标 Bean 实现一个或多个接口时，Spring 默认使用 JDK 动态代理 [p.30]
- **CGLIB 字节码代理**：如果目标类没有实现接口，或设置了 `proxyTargetClass=true`，Spring 使用 CGLIB 创建子类代理 [p.30]
 - 注意：final 类以及 final/private 方法不能被 CGLIB 代理，通知不会织入 [p.30]
- Spring 7 引入 `@Proxyable` 注解，用于设置全局代理策略 [p.30]

3.3 切点表达式 (Pointcut Expressions)

- Spring AOP 使用 AspectJ 的切点表达式语法 [p.32]
- **execution**：匹配方法执行，最常用的设计器（Designator） [p.32]
 - 格式：execution(修饰符? 返回类型声明类型? 方法名 (参数列表) throws?)
 - 示例：execution(* com.example.service.*(..)) 匹配指定包下所有方法 [p.32]
- **within**：限制切面只在某个包或类内生效 [p.32]
 - 示例：within(com.example.service..*) [p.32]
- **args**：按参数类型匹配 [p.32]
 - 示例：args(java.lang.String, ...) [p.32]
- **@annotation** / **@within** / **@target**：按注解匹配 [p.32]

- 逻辑运算: `&&` (与)、`||` (或)、`!` (非) [p.32]
- `@Pointcut` 注解定义切点并复用 [p.32]

3.4 通知类型 (Advice Types)

- `@Before` 前置通知: 在目标方法执行前运行, 适合参数校验、前置日志等 [p.33]
- `@AfterReturning` 后置返回通知: 在方法正常返回后运行, 可捕获返回值但不可修改 [p.33]
- `@AfterThrowing` 异常通知: 在方法抛出异常后执行, 只处理连接点抛出的异常 [p.33]
- `@After` 最终通知: 无论是否抛异常, 都在方法结束时执行, 适合资源清理 [p.33]
- `@Around` 环绕通知: 同时在调用前后执行, 适用于事务管理、缓存、性能监控等 [p.33]
 - 能用其他通知解决时, 不要滥用环绕通知 [p.33]
- 多切面执行顺序: 通过 `@Order` 注解 (数值越小优先级越高) 或实现 `Ordered` 接口控制 [p.33]

3.5 实际项目中使用 AOP

- 直接使用切点表达式自动装配时, 控制切面的使用范围不够精确 [p.35]
- 实际项目中一般将切面定义成自定义注解, 精确控制切入位置 [p.36]
- 典型范例: 自定义性能监控 (Performance Monitoring) 切面注解 [p.37–38]
- 自调用导致 AOP 失效问题: 同类方法内部调用 (self-invocation) 时, 调用不经过代理对象, 导致 AOP 失效 [p.39]

3.6 Spring AOP 常用组件/模块

- 事务管理 (Transaction Management): 基于环绕通知实现, `@Transactional` 在方法调用前开启事务, 正常返回后提交, 抛异常时回滚 [p.40]
- 缓存 (Cache): `@Cacheable` 通过环绕通知检查缓存命中, 命中则跳过方法执行返回缓存结果, 否则执行方法并将结果写入缓存 [p.40]
- 异步方法执行 (Async): `@Async` 在代理中将方法执行提交到线程池, 实现异步调用 [p.40]
- 重试与限流 (Retry & Concurrency Limit): `@Retryable`、`@ConcurrencyLimit` 通过切面拦截方法, 执行重试逻辑或限制并发调用数 [p.40]

4 本章小结 (Chapter Summary)

4.1 要点回顾

- AOP 思想概述: 为什么需要 AOP? AOP 的各种概念 (关注点、切面、连接点、切点、通知、织入、动态/静态横切) [p.41]
- AOP 实现方式: 编译期/语言扩展、类加载器、运行时动态代理——各有特点、适用不同场景, 能实现不同类型的 AOP [p.41]
- Spring AOP 使用: 基本语法 (切点表达式、通知注解)、实际项目中的使用方式 (自定义注解、避免自调用问题) [p.41]

- **IoC 与 AOP 协同：**IoC/Bean 容器作为基础，AOP 在表示层（安全、日志）、业务逻辑层（事务）、模型层（DAO）全面应用横切逻辑 [p.42]

4.2 关键对比 (Key Comparisons)

- **动态横切 vs 静态横切：**动态横切在运行时增强方法执行行为（主流），静态横切在编译时修改类结构（少见，仅 AspectJ 支持） [p.12]
- **JDK 动态代理 vs CGLIB 代理：**JDK 代理要求接口（默认），CGLIB 通过子类生成（无接口时）；final 类/方法不能用 CGLIB [p.30]
- **五种通知类型：**前置 (Before)、后置返回 (AfterReturning)、异常 (AfterThrowing)、最终 (After)、环绕 (Around)——环绕最强大但不应滥用 [p.33]